

## SE3090 ASSIGNMENT 1 | PROJECT BRIEF (REVISED)

# SME Inventory, Order & Field-Ops Mobile Platform

Integrated Full-Stack and Agentic AI Application for Sri Lankan SMEs

**IMPORTANT – GUIDANCE DOCUMENT**

This revised brief balances the four primary business components and distributes distinct Agentic AI contributions across all four students, following the official SE3090 requirements and the AutoCare AI sample pattern. Groups must still implement an original solution; this document is guidance only.

## 1. Project Description

This project builds a unified inventory and sales-order system tailored to Sri Lankan small and medium enterprises (retail, wholesale, light manufacturing, distribution). Field staff currently lack real-time stock visibility; order status is opaque; and managers cannot enforce simple business rules without spreadsheets or disconnected apps. The solution provides role-based web administration, a Flutter field-operations client with device features (QR/barcode and GPS), and an agent-assisted reorder recommendation workflow that requires manager approval.

The four-component ownership model maps cleanly onto the SE3090 individual marking rubric and demonstrates the mobile-first, secure full-stack capability valued by Colombo product and outsourcing companies.

## 2. Real-World Problem Statement

SMEs form a large part of the Sri Lankan economy yet still rely on spreadsheets or disconnected mobile apps for inventory and order management. Field sales staff cannot see current stock levels while visiting customers; orders are confirmed by telephone or messaging apps; low-stock situations are discovered too late; and managers have no reliable audit trail or rule enforcement. The resulting stock-outs, overselling and administrative rework reduce both revenue and operational trust.

### User Roles

1. **Admin** — Manages master data, users, roles and system settings.
2. **Storekeeper** — Performs stock adjustments, manages stock levels and movement history.
3. **Field Sales** — Captures orders on mobile, checks stock, records visits and device captures.
4. **Manager** — Reviews AI reorder proposals, approves or rejects them, views reports and agent workflow.

*Implementation note. User management should be implemented as a shared mandatory feature. It is safer not to count it as one of the four main business components because authentication and role-based authorization are already compulsory requirements.*

## 3. Project Objectives

- Build a unified inventory + sales-order system with role-based web administration and mobile field operations within the 9-week window.
- Provide real-time stock levels, low-stock alerts and a complete agent-assisted reorder recommendation workflow that pauses for manager approval.
- Demonstrate secure token handling on mobile, at least one meaningful device feature (camera/QR or GPS), and shared business rules enforced solely on the ASP.NET Core backend.
- Integrate at least one mandatory third-party service (SMS or email notifications) routed exclusively through the ASP.NET Core API, with timeout and failure recovery.
- Ship a runnable Android APK, deployed API and React dashboard, automated tests, CI pipeline and clear individual contribution evidence.
- Satisfy the mandatory cross-platform evidence pattern: an order may originate on Flutter, persist through ASP.NET Core and PostgreSQL, trigger the agent workflow, receive approval in React, and return status to the mobile user.

## 4. Recommended Technology Stack

| Layer         | Technology                                                                                                                                          |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend       | ASP.NET Core Web API, EF Core, JWT authentication, role claims (Admin, Storekeeper, Field Sales, Manager), DTOs, service layer, validation, Swagger |
| Database      | PostgreSQL – products, stock movements, orders, customers, agent workflow tables; migrations; audit fields; transactional stock updates             |
| Web Client    | React dashboard for inventory management, order approval, reporting and agent monitoring                                                            |
| Mobile Client | Flutter – field order capture, stock check, QR/barcode or camera integration, GPS, secure token storage                                             |
| Agentic AI    | Four specialised agents (Coordinator, Inventory Analyst, Reorder Advisor, Validator) with shared durable state and allow-listed tools               |
| Third-party   | SMS or email notification gateway (free/sandbox tier) for order confirmation and low-stock alerts, routed only through ASP.NET Core                 |
| Engineering   | GitHub Actions CI, unit/widget tests, Swagger, environment-based configuration, ADRs                                                                |

## 5. Key Features & Functionality

- Multi-role system: Admin (master data), Storekeeper (stock adjustments), Field Sales (mobile orders), Manager (approvals).
- Stock movements with history, low-stock thresholds, and basic utilisation reports.
- Order lifecycle (create → confirm → fulfil / cancel) with order lines, status workflow, search/filter/pagination.
- Order creation on Flutter with offline-tolerant form validation and secure token storage.
- Meaningful device features: QR/barcode scan for product lookup and GPS for visit/delivery location capture.
- Mandatory third-party integration: SMS or email notifications for order status and low-stock alerts (credentials protected, failures handled).
- Agentic workflow: “Suggest reorder quantities for items below threshold” → plan → tool calls (current stock, sales velocity) → schema validation → manager approval in React → update stock plan.
- End-to-end evidence: order started on mobile → persisted → agent recommendation → approval on web → status visible on mobile.
- Server-side validation, CORS, structured logging, global exception handling, transactional integrity for stock mutations.

## 6. Technical Complexity Indicators (Medium Level)

- Transactional stock updates and proper foreign-key constraints with audit trails.
- Flutter state management (Provider/Riverpod/Bloc or justified alternative) and secure storage of JWT.
- At least four specialised agents with distinct responsibilities and controlled tools.
- Deterministic business-rule validation before any stock or order mutation is accepted.
- Mandatory third-party notification service with timeout, failure recovery and least-privilege access.
- CI pipeline that builds the API and runs automated tests; clear Git contribution history.

## 7. Four Primary Business Components (One per Student)

Aligned with SE3090 Section 3 / 4.1: each of the four group members owns one major business component with its own entities, API endpoints, React/Flutter surfaces, tests and a **distinct Agentic AI contribution**. Agents are deliberately distributed so no single student owns the entire AI subsystem.

| Component                                      | Main functionality                                                             | Business-specific operation                                          | Primary Agent                  |
|------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------|--------------------------------|
| A – Product & Inventory Management             | Products, categories, stock levels, movements, thresholds and adjustments      | Transactional stock updates and low-stock detection                  | Inventory Analyst Agent        |
| B – Order Management                           | Sales orders, order lines, status history, customers and confirmation workflow | Order lifecycle with stock reservation and status transitions        | Demand / Order-Context Agent   |
| C – Field Sales / Mobile Operations            | Field agent profiles, visits, device captures (QR/GPS) and offline sync        | Mobile order capture with device features and offline tolerance      | Field Context Agent            |
| D – Reorder Planning, Approval & Notifications | Agent workflows, reorder proposals, validation, approvals and notifications    | Agent-driven reorder plan with manager approval gate + notifications | Coordinator + Validator Agents |

Each student must implement for their component:

- ASP.NET Core API endpoints
- PostgreSQL entities and relationships
- React interfaces
- Flutter interfaces
- Tests and documentation
- Distinct Agentic AI contribution (see agent ownership below)

### Example Complete Scenario

A field sales agent submits the following objective using Flutter:

“Check stock for Product SKU-4521 at customer site, place an order for 40 units if available, and flag for reorder if current warehouse stock falls below threshold.”

The agent scans the product QR code, captures GPS location of the visit, and submits the order request with preferred delivery window.

#### Workflow

- **1. Flutter submission.** The field agent logs in and submits the order/stock-check request with product selection (via QR), quantity, customer, location and optional notes.
- **2. ASP.NET Core processing.** The API validates the JWT, Field Sales role, product existence, stock availability rules, and saves the request in PostgreSQL.
- **3. Coordinator / Planning Agent (Student 4).** Analyses the objective and creates a structured plan: retrieve current stock, examine sales velocity, check thresholds, propose reorder quantities, validate the proposal, request manager approval.
- **4. Inventory Analyst Agent (Student 1).** Examines stock levels, movement history and thresholds; supplies structured stock and velocity data via allow-listed tools.
- **5. Demand / Order-Context Agent (Student 2).** Supplies recent order history, customer patterns and line-item aggregates to inform demand signals.
- **6. Field Context Agent (Student 3).** Enriches the plan with visit location, GPS capture and device-capture metadata when relevant for validation.
- **7. Reorder Advisor + Validator (Student 4).** Calculates recommended reorder quantities; performs schema and business-rule validation. Cannot mutate stock until manager approves.
- **8. Deterministic validation.** Before accepting the AI result the system verifies: all products exist, stock calculations are correct, thresholds are respected, output follows required JSON structure.

- **9. Human approval.** Workflow status becomes *PendingManagerApproval*. Manager uses the React dashboard to review the plan, stock impact, validation results and execution history → Approve / Reject / Request revision.
- **10. Final execution.** After approval, ASP.NET Core uses a database transaction to apply reorder adjustments (or confirm order), update stock and finalise the plan.
- **11. Flutter status update.** Field agent receives order confirmation / stock alert status and any approved reorder outcome through the mobile application.

**Complete cross-platform workflow:**

**Flutter → ASP.NET Core → PostgreSQL → Agentic AI → React approval → Flutter status update**

## 8. Four Specialised Agents (Distributed Ownership)

Each agent has a clearly defined responsibility, structured input/output, controlled tool permissions, error handling, visible participation in the workflow, tests and documentation. Ownership is split so every student has a viva-defendable Agentic AI contribution.

| Agent                              | Owner                   | Responsibility                                                                                                                                            |
|------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Coordinator / Planning Agent       | Student 4 (Component D) | Converts the customer/field objective into a structured multi-step plan and delegates tasks to specialised agents                                         |
| Inventory Analyst Agent            | Student 1 (Component A) | Supplies current stock, movement velocity and threshold data; performs pre-validation of stock availability via allow-listed tools                        |
| Demand / Order-Context Agent       | Student 2 (Component B) | Supplies order history, customer patterns and line-item aggregates; participates in demand-signal analysis for reorder suggestions                        |
| Field Context Agent                | Student 3 (Component C) | Enriches the workflow with location/visit/device-capture metadata; supplies structured GPS/QR data for validation of delivery or stock-check claims       |
| Reorder Advisor + Validator Agents | Student 4 (Component D) | Calculates recommended reorder quantities; enforces schema and deterministic business-rule checks before any mutation; never mutates stock until approval |

**Note on agent count:** The Coordinator, Inventory Analyst, Demand/Order-Context, Field Context and Validator/Reorder-Advisor roles together satisfy the “at least four distinct agents” rule. Student 4 owns the orchestration + final validation + approval gate (the high-impact action that must pause for human approval). Students 1–3 each own a specialised analysis agent that is called by the Coordinator and appears in the execution summary.

## 9. React Web Application

The React application will be used mainly by staff for:

- Managing products, categories and stock levels
- Managing stock movements, thresholds and utilisation reports
- Order desk, status workflow, search/filter/pagination
- Field-agent performance overview and visit history
- Dashboards and reports (low-stock, order status)
- Agent workflow monitoring and execution summaries
- Approving, rejecting or revising AI reorder proposals
- Viewing audit, notification and execution history

## 10. Flutter Mobile Application

The Flutter application will support field sales staff through:

- Registration, login and logout
- Secure JWT token storage
- Product lookup (including QR/barcode scan)
- Current stock display and simple movement history
- Order capture form with offline-tolerant validation
- GPS location capture for visits / delivery
- Customer lookup and visit recording
- Order status list and detail view
- Notifications (order confirmation, stock alerts)
- Protected mobile routes and offline sync queue

This provides meaningful mobile device features such as the camera (QR/barcode), GPS and secure storage.

## 11. Third-Party Integration

An SMS or email notification gateway is integrated through ASP.NET Core to:

- Send order confirmation and status updates
- Notify managers / storekeepers of low-stock situations
- Support optional delivery or visit-related alerts

The backend must protect API credentials and handle timeouts, invalid responses, rate limits and service failures. All third-party calls are routed exclusively through the ASP.NET Core API.

## 12. Detailed Component Ownership

### Component A — Product & Inventory Management

**Owner:** Student 1

**Core Entities:** Product, ProductCategory, StockLevel, StockMovement, LowStockThreshold, StockAdjustment.

**API Endpoints (examples):** GET/POST/PUT/DELETE /api/products, GET /api/products/search (filter, sort, pagination), POST /api/stock/movements, GET /api/stock/levels, PUT /api/stock/thresholds, GET /api/stock/history.

**React Surface:** Product catalogue administration, stock level dashboard, movement history, threshold configuration, utilisation reports.

**Flutter Surface:** Product lookup (including QR scan), current stock display for a selected product, simple movement history for field awareness.

**Agentic AI Contribution:** *Inventory Analyst Agent* – supplies current stock, movement velocity and threshold data to the reorder workflow via allow-listed tools; performs pre-validation of stock availability. Must have its own input/output contract, error handling and appear in the workflow execution summary.

**Inter-Component Dependencies:** Provides product and stock identity to Component B (Orders) and Component C (Field Ops). Consumes approved reorder plans from Component D that adjust stock levels.

### Component B — Order Management

**Owner:** Student 2

**Core Entities:** SalesOrder, OrderLine, OrderStatusHistory, Customer, PaymentReference (optional).

**API Endpoints (examples):** POST /api/orders, GET /api/orders (filter/sort/paginate by status, customer, date, field agent), PUT /api/orders/{id}/status, GET /api/orders/{id}/lines, POST /api/orders/{id}/confirm, POST /api/orders/{id}/cancel.

**React Surface:** Order desk, status workflow controls, search/filter panels, order detail and line-item views, confirmation/cancellation actions.

**Flutter Surface:** Order capture form (with offline-tolerant validation), order status list, order detail view, cancellation request.

**Agentic AI Contribution:** *Demand / Order-Context Agent* – supplies order history, customer patterns and line-item aggregates to the planning workflow; participates in demand-signal analysis for reorder suggestions. Distinct contract and visible participation required.

**Inter-Component Dependencies:** Depends on Component A for product validity and stock reservation. Status changes trigger notifications owned by Component D. Field capture originates from Component C.

## Component C — Field Sales / Mobile Operations

**Owner:** Student 3

**Core Entities:** FieldAgentProfile, CustomerVisit, DeviceCapture (QR/GPS payload), OfflineSyncQueue.

**API Endpoints (examples):** GET /api/field/customers, POST /api/field/visits, POST /api/field/captures, GET /api/field/my-orders, POST /api/field/sync, GET /api/field/stock-check/{productId}.

**React Surface:** Field-agent performance overview, visit history map/list, device-capture audit (admin view), sync-status monitoring.

**Flutter Surface:** Primary field UX – login with secure token storage, customer lookup, QR/barcode or GPS capture, order entry forms, offline-tolerant queue, protected mobile routes.

**Agentic AI Contribution:** *Field Context Agent* – enriches the agent workflow with location or visit metadata when relevant; supplies structured device-capture data for validation of delivery or stock-check claims. Must be callable by the Coordinator and logged in the execution summary.

**Inter-Component Dependencies:** Initiates orders that flow into Component B. Performs stock lookups against Component A. Receives final order and notification status from Component D.

## Component D — Reorder Planning, Approval & Notifications

**Owner:** Student 4

**Core Entities:** AgentWorkflow, ReorderProposal, WorkflowStep, ToolCallLog, ValidationResult, NotificationLog, ApprovalGate.

**API Endpoints (examples):** POST /api/agent/reorder (start), GET /api/agent/workflows/{id}, POST /api/agent/workflows/{id}/approve, POST /api/agent/workflows/{id}/reject, GET /api/agent/workflows/{id}/summary, POST /api/notifications/send, GET /api/reports/low-stock.

**React Surface:** Manager approval UI for reorder proposals, agent execution summaries, notification audit log, low-stock report dashboard.

**Flutter Surface:** Display of approved reorder outcomes (where relevant to field staff), notification of order confirmation or stock alerts, workflow status indicator.

**Agentic AI Contribution:** *Coordinator / Planning Agent + Reorder Advisor Agent + Validator Agent* – full minimum-acceptance workflow ownership for planning, tool orchestration, schema/business-rule validation, human-approval gate, safe failure, and the mandatory third-party SMS/email integration. This student owns the high-impact action that must pause for authorised approval.

**Inter-Component Dependencies:** Consumes stock data (A), order history (B) and field context (C). Writes approved reorder adjustments back to stock levels (A) and emits notifications consumed by both clients.

## 13. Suggested 9-Week Timeline

| Weeks | Focus                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------------|
| 1–2   | Domain modelling, ER diagram, ADR, core API + PostgreSQL schema, JWT setup, component ownership agreed, agent contracts defined. |
| 3–4   | React inventory & order screens; Flutter field app with login, product lookup and secure token storage.                          |
| 5–6   | Device feature (QR/GPS), stock movement logic, agentic reorder workflow (all four agents), third-party SMS/email integration.    |
| 7–9   | Testing (unit, integration, agent golden cases), CI, deployment, APK generation, documentation, demonstration video.             |

### Final reminder

This document is guidance only. Each group must implement an original solution that satisfies every mandatory requirement in the official SE3090 Assignment 1 specification (technology stack, four components, four distinct agents, human approval gate, cross-platform workflow, tests, CI, deployment and viva readiness).